



UNIVERSIDADE DA CORUÑA



facultade de
informática
da coruña

Práctica 1: Aprendizaje por refuerzo y redes neuronales

Gymnasium, Stablebaselines3 y RoboboSim

Grado en Inteligencia Artificial. Robótica Inteligente Aplicada

Herramientas. Qué aporta cada una

Ninguna resuelve la práctica por sí sola: cada una cubre una capa distinta.

RoboSim	Módulo sim	ROS 2 + robobo_ros2	Gymnasium	Stable-Baselines3
QUÉ ES Simulador del Robobo hecho con Unity: física, sensores y escena.	QUÉ ES Parte del nodo puente que expone el simulador en ROS 2, bajo sim/.	QUÉ ES Middleware de robótica y el nodo puente que expone el Robobo en él.	QUÉ ES Estándar de interfaz para entornos de aprendizaje por refuerzo.	QUÉ ES Implementaciones fiables de algoritmos de RL sobre PyTorch.
EN LA PRÁCTICA Sustituye al robot real mientras se entrena. Mismos sensores, misma interfaz.	EN LA PRÁCTICA Reiniciar la escena, colocar el robot y conocer su posición real. Sólo en simulación.	EN LA PRÁCTICA Leer sensores (topics) y dar órdenes (cmd_vel, servicios). Igual en sim y en real.	EN LA PRÁCTICA Define qué es una observación, una acción y un episodio: reset() y step().	EN LA PRÁCTICA SAC y PPO: aprenden la política a partir de lo que devuelve step().
<i>Equipo anfitrión</i>	<i>Contenedor</i>	<i>Contenedor</i>	<i>Contenedor</i>	<i>Contenedor</i>

Dónde se ejecuta: sólo el simulador corre en el equipo anfitrión (Windows o macOS). Todo lo demás está instalado en el contenedor Docker, que alcanza al simulador en host.docker.internal.

<https://gymnasium.farama.org/>

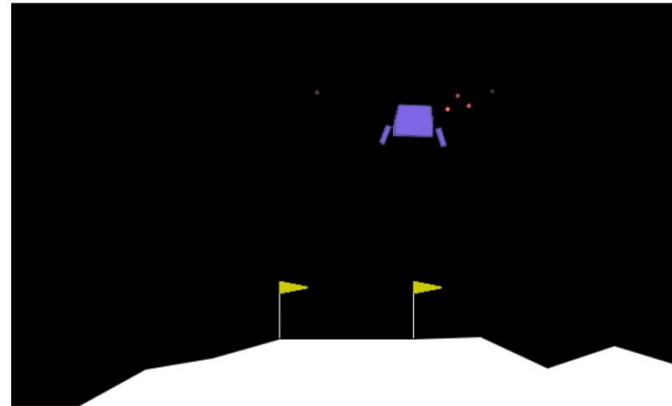


Instalar Gymnasium

```
pip install "gymnasium[all]"
```

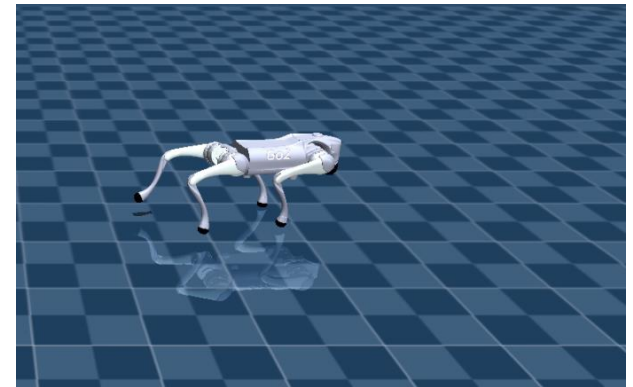
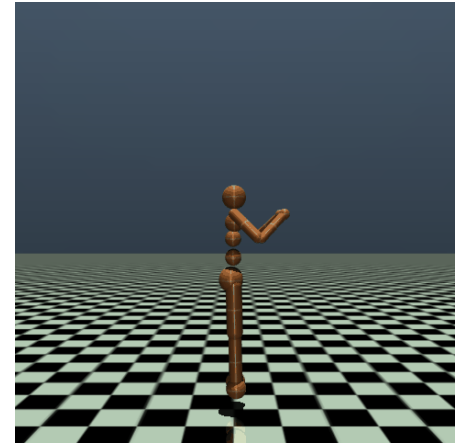
An API standard for reinforcement learning with a diverse collection of reference environments

<https://pypi.org/project/gymnasium/>



Gymnasium. Estructura básica. ¿Que es un entorno?

- Un entorno de Gymnasium es un mundo simulado en el que un agente interactúa con su ambiente (y generalmente aprende).
- Entre otros, encapsula la dinámica de la tarea:
 - Qué puede observar el agente (el **espacio de observación**).
 - Qué acciones puede realizar el agente (el **espacio de acción**).
 - Cómo evoluciona el entorno en respuesta a las acciones (la **dinámica de transición**).
 - Cómo se recompensa al agente por su comportamiento (la **función de recompensa**).



▪ 1.- Pasos para usar Gymnasium:

- Definir entorno con `make()`(No es necesario pero es recomendable)
- En el entorno:
 - **`reset()`** al principio de todo y al principio de cada episodio.
 - Mientras no concluya el episodio:
 - Escoger la acción a ejecutar.
 - **`step()`** para indicar la acción y recibir la sensorización y la recompensa.
 - **`render()`** para visualizar el entorno (si el valor indicado para `render_mode` en `make()` es “human”, `render()` se llama automáticamente en `step()`).
 - **`close()`** para cerrar el entorno (ventanas, BDs, conexiones HTTP...)

Más información: https://gymnasium.farama.org/introduction/basic_usage/

Gymnasium. Estructura básica. Ejemplo

- https://gymnasium.farama.org/introduction/basic_usage/

```
# Run `pip install "gymnasium[classic-control]"` for this example.
import gymnasium as gym

# Create our training environment - a cart with a pole that needs balancing
env = gym.make("CartPole-v1", render_mode="human")

# Reset environment to start a new episode
observation, info = env.reset()

# observation: what the agent can "see" - cart position, velocity, pole angle, etc.
# info: extra debugging information (usually not needed for basic learning)

print(f"Starting observation: {observation}")
# Example output: [ 0.01234567 -0.00987654  0.02345678  0.01456789]
# [cart_position, cart_velocity, pole_angle, pole_angular_velocity]

episode_over = False
total_reward = 0

while not episode_over:
    # Choose an action: 0 = push cart left, 1 = push cart right
    action = env.action_space.sample() # Random action for now - real agents will be smarter!

    # Take the action and see what happens
    observation, reward, terminated, truncated, info = env.step(action)

    # reward: +1 for each step the pole stays upright
    # terminated: True if pole falls too far (agent failed)
    # truncated: True if we hit the time limit (500 steps)

    total_reward += reward
    episode_over = terminated or truncated

print(f"Episode finished! Total reward: {total_reward}")
env.close()
```

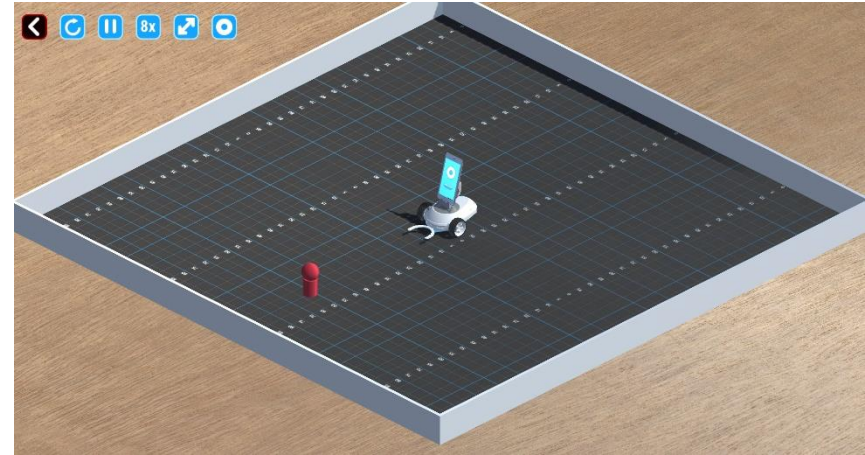
Gymnasium. Estructura básica. ¿Cómo crear un entorno personalizado?

- https://gymnasium.farama.org/introduction/create_custom_env/

1. ¿Qué tiene que aprender el agente?
2. ¿Qué información necesita?
3. ¿Qué acciones puede realizar?
4. ¿Cómo evaluamos el éxito (recompensa)?
5. ¿Cuándo debe de terminar un episodio?

Particularizarlo a nuestra práctica: Aprendizaje por refuerzo con el simulador RoboSim + ROS2

Episodio: Secuencia de interacciones entre el agente y el entorno, que comienza desde un estado inicial y termina cuando se alcanza un estado terminal (o condición de detención).



Gymnasium. Estructura básica. Entorno personalizado con RoboSim

- https://gymnasium.farama.org/introduction/create_custom_env/
- Pasos a seguir:
 1. Enlazar RoboSim + ROS2 + Gymnasium
 2. Definir el espacio de observaciones
 3. Definir el espacio de acciones
 4. Definir la función de recompensa
 5. Definir los métodos básicos de cualquier entorno de Gymnasium:
 1. Reset()
 2. Step()
 3. Close()
 6. Verificar la creación del entorno.

```
# Run "pip install "gymnasium[classic-control]" for this example.
import gymnasium as gym

# Create our training environment - a cart with a pole that needs balancing
env = gym.make("CartPole-v1", render_mode="human")

# Reset environment to start a new episode
observation, info = env.reset()
# observation: what the agent can "see" - cart position, velocity, pole angle, etc.
# info: extra debugging information (usually not needed for basic learning)

print(f"Starting observation: {observation}")
# Example output: [ 0.01234567 -0.00987654  0.02345678  0.01456789]
# [cart_position, cart_velocity, pole_angle, pole_angular_velocity]

episode_over = False
total_reward = 0

while not episode_over:
    # Choose an action: 0 = push cart left, 1 = push cart right
    action = env.action_space.sample() # Random action for now - real agents will be smarter!

    # Take the action and see what happens
    observation, reward, terminated, truncated, info = env.step(action)

    # reward: +1 for each step the pole stays upright
    # terminated: True if pole falls too far (agent failed)
    # truncated: True if we hit the time limit (500 steps)

    total_reward += reward
    episode_over = terminated or truncated

print(f"Episode finished! Total reward: {total_reward}")
env.close()
```


Stablebaselines3. Instalación

<https://stable-baselines3.readthedocs.io/en/master/>

Instalar Gymnasium

```
pip install stable-baselines3[extra]
```

<https://stable-baselines3.readthedocs.io/en/master/guide/install.html>



Stablebaselines3. Entrenar en un entorno existente

```
import gymnasium as gym

from stable_baselines3 import A2C

env = gym.make("CartPole-v1", render_mode="rgb_array")

model = A2C("MlpPolicy", env, verbose=1)
model.learn(total_timesteps=10_000)

vec_env = model.get_env()
obs = vec_env.reset()
for i in range(1000):
    action, _state = model.predict(obs, deterministic=True)
    obs, reward, done, info = vec_env.step(action)
    vec_env.render("human")
    # VecEnv resets automatically
    # if done:
    #     obs = vec_env.reset()
```

Stablebaselines3. Entrenar en un entorno “custom”

[https://stable-baselines3.readthedocs.io/en/master/guide/custom_e
nv.html](https://stable-baselines3.readthedocs.io/en/master/guide/custom_env.html)

```
import gymnasium as gym
import numpy as np
from gymnasium import spaces

class CustomEnv(gym.Env):
    """Custom Environment that follows gym interface."""

    metadata = {"render_modes": ["human"], "render_fps": 30}

    def __init__(self, arg1, arg2, ...):
        super().__init__()
        # Define action and observation space
        # They must be gym.spaces objects
        # Example when using discrete actions:
        self.action_space = spaces.Discrete(N_DISCRETE_ACTIONS)
        # Example for using image as input (channel-first; channel-last also works):
        self.observation_space = spaces.Box(low=0, high=255,
                                             shape=(N_CHANNELS, HEIGHT, WIDTH), dtype=np.uint8)

    def step(self, action):
        ...
        return observation, reward, terminated, truncated, info

    def reset(self, seed=None, options=None):
        ...
        return observation, info

    def render(self):
        ...

    def close(self):
        ...
```

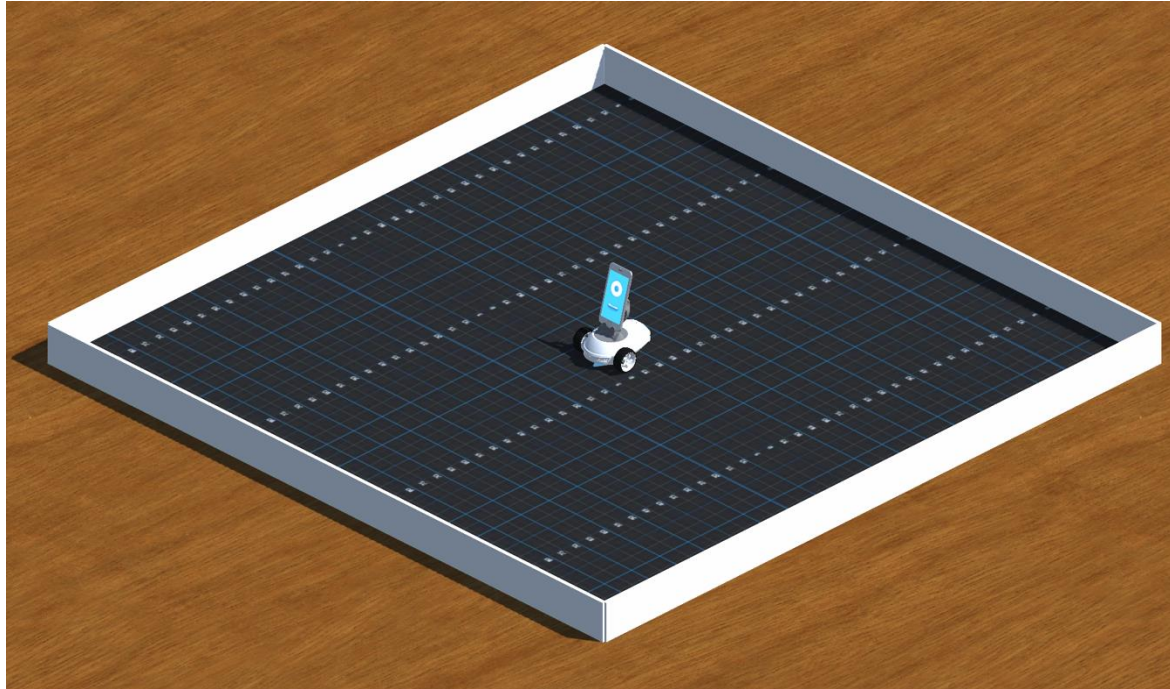
Gymnasium y Stablebaselines3. “Tips and tricks”

https://stable-baselines3.readthedocs.io/en/master/guide/rl_tips.html

Consejos y trucos al crear un entorno personalizado:

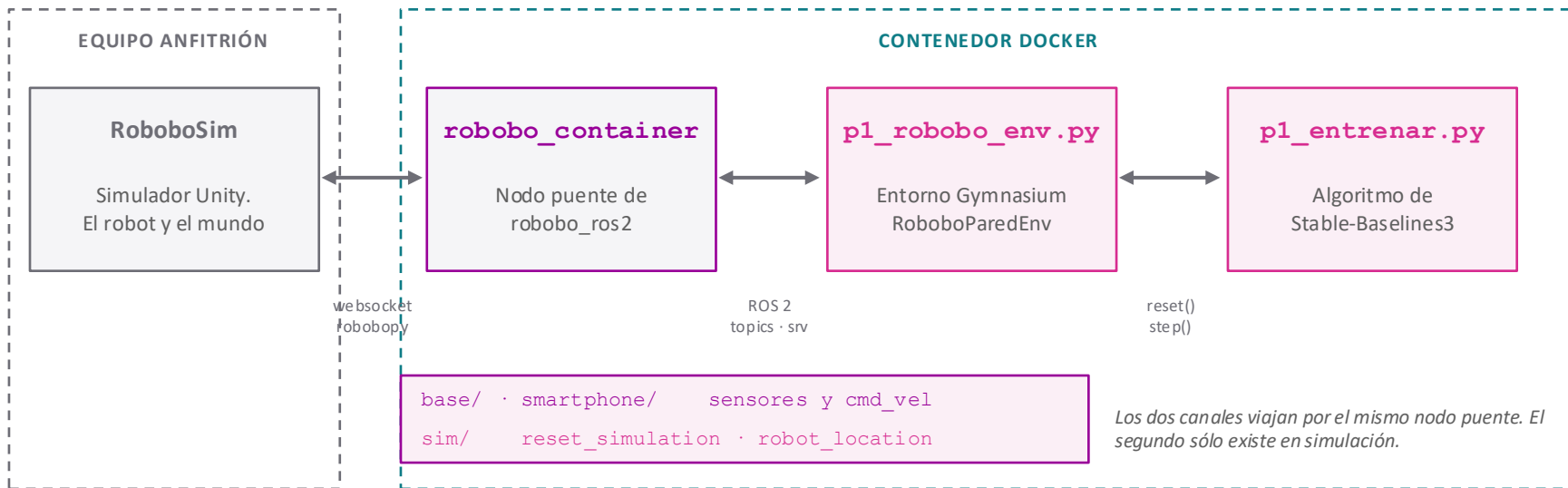
1. **Normaliza** siempre tu espacio de observación si es posible.
2. **Normaliza** tu espacio de acciones. Una buena práctica es reescalar tus acciones para que se encuentren en $[-1, 1]$. Esto no te limita, ya que puedes reescalar fácilmente la acción dentro del entorno.
3. Comienza con una **recompensa informativa** y una **versión simplificada** de tu problema.
4. **Depura con acciones aleatorias** para comprobar si tu entorno funciona y sigue la interfaz del gimnasio.

Tarea: aproximarse a una pared y detenerse cerca de ella sin llegar a chocar



Práctica 1. Las piezas del programa de ejemplo

Dos archivos propios sobre cuatro herramientas. Cada pieza habla sólo con sus vecinas.



Lo que percibe y hace el robot

Llega y sale por ROS 2. Es lo único que existe también en el robot real.

El canal sim/ sólo toca el mundo

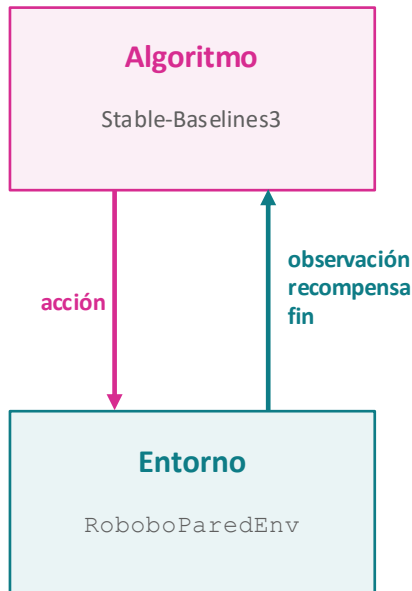
reinicia la escena y da la posición real. Nunca informa a la política.

El algoritmo no sabe de robots

sólo ve `reset()` y `step()`. Todo lo demás lo esconde el entorno.

Gymnasium. El contrato del entorno

El algoritmo y el robot sólo se comunican a través de dos métodos.



```
class RoboboParedEnv(gym.Env):  
  
    observation_space = Box(...) # qué ve  
    action_space      = Box(...) # qué hace  
  
    def reset(seed, options):  
        ... # empezar episodio  
        return obs, info  
  
    def step(accion):  
        ... # un paso de control  
        return obs, r, terminated, truncated, info  
  
    def close(): ...
```

terminated

El episodio acaba por lo que ha pasado: choque o tarea resuelta.

truncated

Se corta por tiempo: 200 pasos. No es ni éxito ni fracaso.

Stable-Baselines3. SAC y PPO

Los dos admiten acciones continuas. Lo que los distingue es cómo aprovechan cada paso.

SAC

fuera de política · por defecto en el ejemplo

- Guarda cada transición en un búfer y la reutiliza muchas veces.
- Aprende más con menos pasos del robot.
- Maximiza también la entropía: explora mejor.
- `learning_starts = 500`: pasos aleatorios antes de aprender.

PPO

dentro de política · --algoritmo ppo

- Usa cada lote de pasos una sola vez y lo descarta.
- Más estable y fácil de ajustar, pero necesita más pasos.
- Explora con ruido que se va reduciendo.
- `n_steps = 512`: el defecto, 2048, serían diez minutos sin actualizar.

USO

```
modelo = SAC('MlpPolicy', env)
modelo.learn(total_timesteps=10000)
modelo.save('p1_modelo_sac')

modelo = SAC.load('p1_modelo_sac')
a, _ = modelo.predict(obs,
                      deterministic=True)
```

El mismo código sirve para PPO cambiando SAC por PPO. El modelo guardado es un .zip que se carga sin reentrenar.

Por qué importa aquí. Cada paso del entorno cuesta unos 0,3 s de tiempo real: 10 000 pasos son del orden de 50 minutos. Los valores por defecto de Stable-Baselines3 suponen miles de pasos por segundo, por eso el ejemplo ajusta los hiperparámetros que dependen del número de pasos.

p1_robobo_env.py. Estructura del archivo

Una clase, RoboboParedEnv, organizada en bloques. Tarea: acercarse a una pared y detenerse sin chocar.

Constantes	<code>NS_BASE, IR_SATURACION, P_OBJETIVO, P_CHOQUE, V_MAX, W_MAX, PASO_S...</code>
<code>__init__</code>	Crea el nodo ROS 2, el publicador de <code>cmd_vel</code> , la suscripción a <code>ir</code> , el cliente de <code>sim/reset_simulation</code> y los espacios
Comunicación	<code>_cb_ir · _vaciarCola · _esperar_lectura_nueva · _publicar_velocidad · _detener · _reiniciar_escena</code>
Observación	<code>_observacion · _proximidad_frontal</code>
Recompensa	<code>_recompensa</code> ← la parte que se estudia y se modifica
Gymnasium	<code>reset · step · close</code>
Utilidad	<code>calibrar()</code> : imprime los infrarrojos frontales en continuo

CONSTANTES QUE HAY QUE CALIBRAR

<code>IR_SATURACION</code>	1000
<code>P_OBJETIVO</code>	0,55
<code>P_TOLERANCIA</code>	0,08
<code>P_CHOQUE</code>	0,85
<code>V_MIN / V_MAX</code>	-0,08 / 0,15 m/s
<code>W_MAX</code>	1,0 rad/s
<code>PASO_S</code>	0,2 s

Dependen del simulador y del robot: se miden con `python3 p1_entrenar.py --calibrar` antes de entrenar.

p1_robobo_env.py. Observación, acción y recompensa

Lo que ve la política, lo que puede hacer y lo que se le premia.

Observación

Box de dimensión 7

[0..4]

5 infrarrojos frontales, divididos por IR_SATURACION y recortados a [0, 1]

[5, 6]

velocidad lineal y angular ordenadas en el paso anterior

Todo viene de sensores del robot, por ROS 2.

Acción

Box(-1, 1) de dimensión 2

[0]

velocidad lineal, reescalada a [-0,08; 0,15] m/s

[1]

velocidad angular, reescalada a [-1; 1] rad/s

Se publica como Twist en cmd_vel.

Recompensa

por paso y al terminar

$-|p - 0,55|$

acercarse a la proximidad objetivo

$-0,05 \cdot |w|$

no girar sin necesidad

-10

choque: $p \geq 0,85$ (termina)

+10

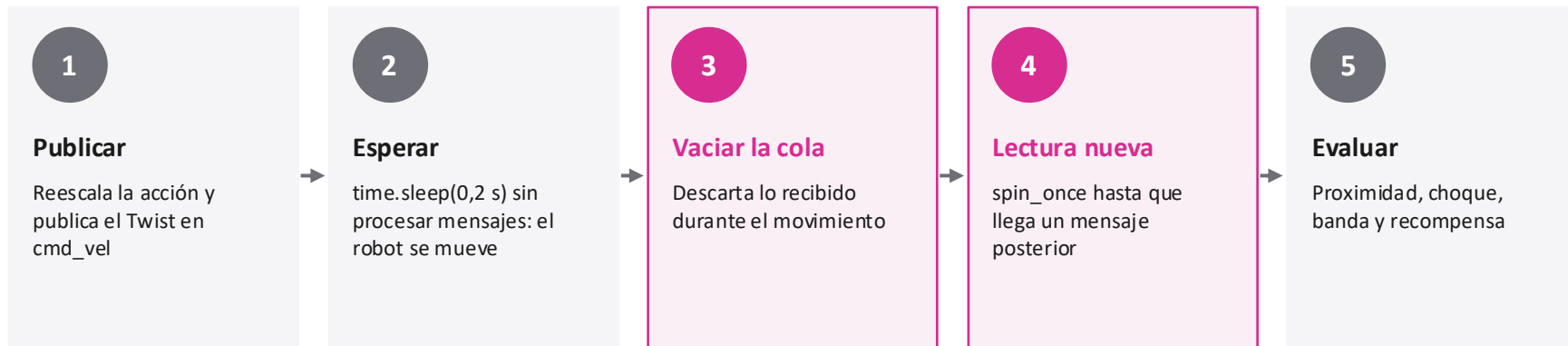
10 pasos seguidos en la banda (termina)

p = mayor proximidad de los cinco frontales.

Episodio. Como máximo 200 pasos de 0,2 s. Termina antes por choque o por tarea resuelta. reset() detiene el robot, llama al servicio si m/reset_simulation y espera una lectura nueva antes de devolver la primera observación.

p1_robobo_env.py. Qué ocurre en un step()

El punto delicado: Gymnasium pide la observación cuando quiere; ROS 2 la entrega cuando llega.



Por qué los pasos 3 y 4

Si step() devolviera sin más la última lectura guardada, la observación sería de **antes** de la acción. El algoritmo recibiría pares (observación, acción) desplazados un paso.

El error no da ningún mensaje: el entrenamiento corre con normalidad y la política simplemente no aprende.

p1_entrenar.py. Modos de uso

Un único programa con cinco modos, pensados para ejecutarse en este orden.

MODO	QUÉ HACE	QUÉ SE OBTIENE
<code>--calibrar</code>	Imprime los cinco infrarrojos frontales en continuo	Valores para las constantes del entorno
<code>--comprobar</code>	check_env de Stable-Baselines3 sobre el entorno	Aviso si el entorno incumple la interfaz
<code>--aleatorio</code>	Episodios con acciones aleatorias	La línea base que hay que superar
<code>--entrenar</code>	Entrena SAC o PPO durante --pasos pasos	p1_modelo_sac.zip y registros en p1_logs/
<code>--evaluar</code>	Ejecuta la política aprendida, determinista	Recompensa, resueltos y choques por episodio

OPCIONES ÚTILES

```
--algoritmo sac|ppo  --pasos 10000  --episodios 5  --pasos-max 200
--detalle (una línea por paso)  --sin-simulador (reposición a mano)
```

Puesta en marcha. Tres terminales

El simulador fuera del contenedor; el puente y el ejemplo dentro, cada uno en su terminal.

1 · Anfitrión

Abrir RoboSim

Cargar el escenario plano
de cuatro paredes

2 · Contenedor

```
ros2 run robobo_ros2 \  
  robobo_container \  
  --ros-args \  
  -p ip:=host.docker.internal \  
  -p modules=["sim"]
```

3 · Contenedor

```
python3 p1_entrenar.py --calibrar  
python3 p1_entrenar.py --comprobar  
python3 p1_entrenar.py --aleatorio  
python3 p1_entrenar.py --entrenar  
python3 p1_entrenar.py --evaluar
```

Módulos mínimos

El ejemplo usa infrarrojos, cmd_vel y el módulo sim, que es el que reinicia la escena. Cada módulo extra compite por la conexión.

Orden de arranque

Primero el simulador, luego el puente y por último el ejemplo, que espera la primera lectura.

Sugerencias para la P1

- Empezar con una versión simplificada del problema, en la cuál se asegure que Robobo es capaz de aprender “algo” con el algoritmo que se utilice de Stablebaselines3 en el entorno que se ha creado con él en Gymnasium
- Hay que tener en cuenta que el proceso de aprendizaje es un proceso iterativo de mejora, que en muchos casos implica
 - Mejorar/redefinir:
 - Espacio de observación
 - Espacio de acciones
 - Función de recompensa
 - Etc.
- Hay que tener en cuenta que hay que aprender una política, pero esa política se tiene que comprobar que funciona con otro script por separado. En este caso, puede ser recomendable comprobar que después del entrenamiento se puede utilizar dicha política, aunque ésta no haya aprendido nada
- Hay que tener en cuenta que se pide una representación de los datos diferente a la que ofrece el “logger” por defecto de Stablebaselines3. Al igual que en el caso anterior, puede ser recomendable comprobar que después del entrenamiento se pueden leer dichos datos, aunque el aprendizaje haya sido pobre